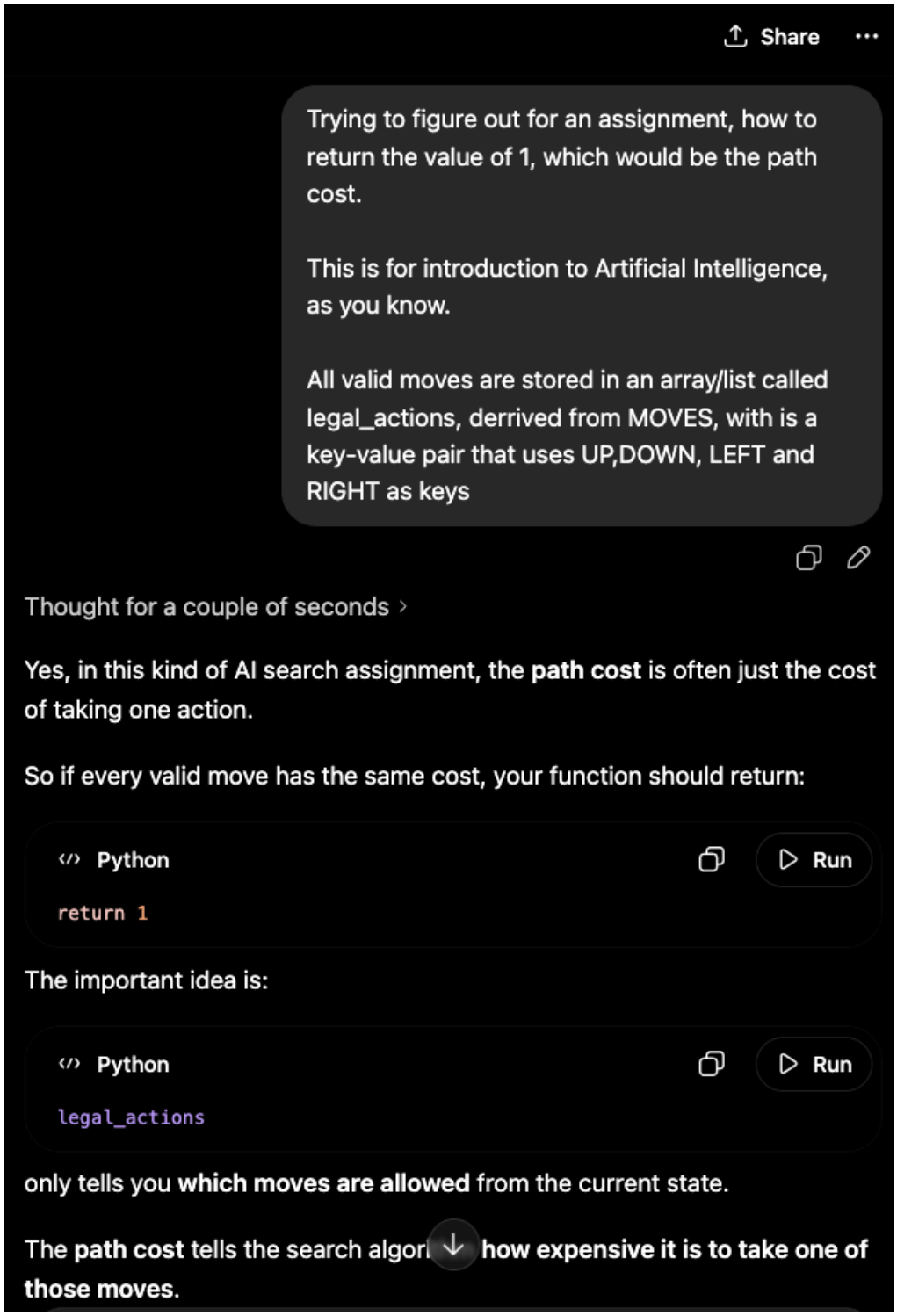
Part A:


Share
Trying to figure out for an assignment, how to return the value of 1, which would be the path cost.
This is for introduction to Artificial Intelligence, as you know.
All valid moves are stored in an array/list called legal_actions, derrived from MOVES, with is a key-value pair that uses UP,DOWN, LEFT and RIGHT as keys
Thought for a couple of seconds
Yes, in this kind of AI search assignment, the path cost is often just the cost of taking one action.
So if every valid move has the same cost, your function should return:
Python
Run
return 1
The important idea is:
Python
Run
legal_actions
only tells you which moves are allowed from the current state.
The path cost tells the search algori
how expensive it is to take one of those moves.

Share ···

The important idea is:

Python Run

```
legal_actions
```

only tells you **which moves are allowed** from the current state.

The **path cost** tells the search algorithm **how expensive it is to take one of those moves**.

So even if your moves come from something like:

Python Run

```
MOVES = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1)
}
```

and your valid moves are stored in:

Python Run

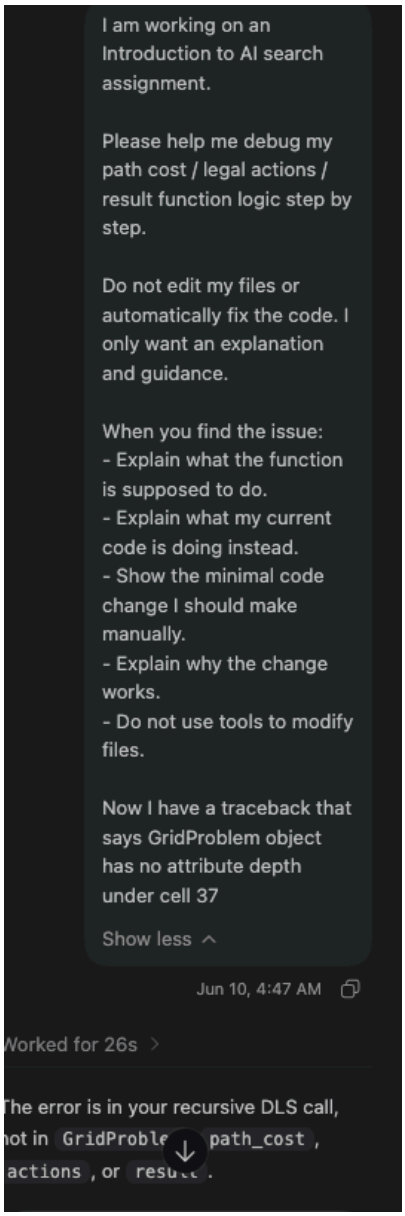
```
legal_actions = ["UP", "RIGHT"]
```

the cost of choosing `"UP"` or `"RIGHT"` can still just be `1`.

A simple version could look like this:

Python Run

```
def path_cost(action):
    return 1
```


I am working on an Introduction to AI search assignment.
Please help me debug my path cost / legal actions / result function logic step by step.
Do not edit my files or automatically fix the code. I only want an explanation and guidance.
When you find the issue:
- Explain what the function is supposed to do.
- Explain what my current code is doing instead.
- Show the minimal code change I should make manually.
- Explain why the change works.
- Do not use tools to modify files.
Now I have a traceback that says GridProblem object has no attribute depth under cell 37
Show less
Jun 10, 4:47 AM
Worked for 26s
The error is in your recursive DLS call,
ot in GridProble path_cost ,
actions , or resu .

Worked for 26s

The error is in your recursive DLS call, not in `GridProblem`, `path_cost`, `actions`, or `result`.

What the function expects

```python
def _recursive_dls(self, proble
```

- `problem` should be a `GridProblem`.
- `node` should be a `Node`, which has `.depth`.

What your code does

```python
result, status = self._recursiv
```

The first two arguments are reversed. Therefore:

- `problem` receives `child`
- `node` receives the `GridProblem`

Then this line fails:

```python
node.depth
```

because `node` is actually the `GridProblem`.

Minimal change

This says the problem was not defined here. But is problem a parameter for this?

Lab_1_Part_A.ipynb • Cell 14:9-10

Optimized tool selection

Analyzed GridProblem class and its actions method

No, `problem` is not a parameter here. The parameter is `state` (a tuple like `(row, col)`).

Looking at the incomplete code in the `actions` method, I see several issues:

```
def actions(self, state: Tuple[int,
    legal_actions = []
    dr, dc = MOVES[state]  # ❌ ERI
    n_row, n_col = self.row + dr, s
```

The problems are:

1. `MOVES[state]` — You can't look up a state (like `(0,0)`) in MOVES. MOVES keys are action **names** like `"UP"`, `"DOWN"`, etc.
2. `self.row` — Should be unpacking the state first: `row, col = state`
3. **Missing loop** — You need to loop through each action in MOVES, not try to index state directly.

Restores workspace and chat to this point

implementation is correct, I got. a bit confused

Lab_1_Part_A.ipynb • Cell 26:6-34

Optimized tool selection

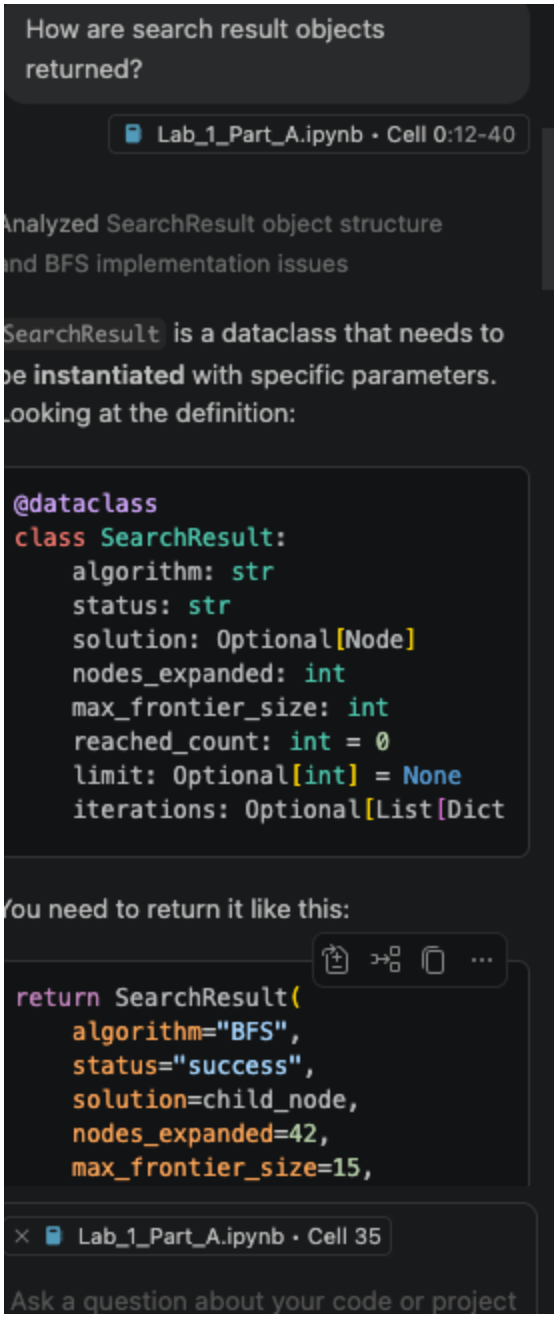
Analyzed BFS implementation issues in search method

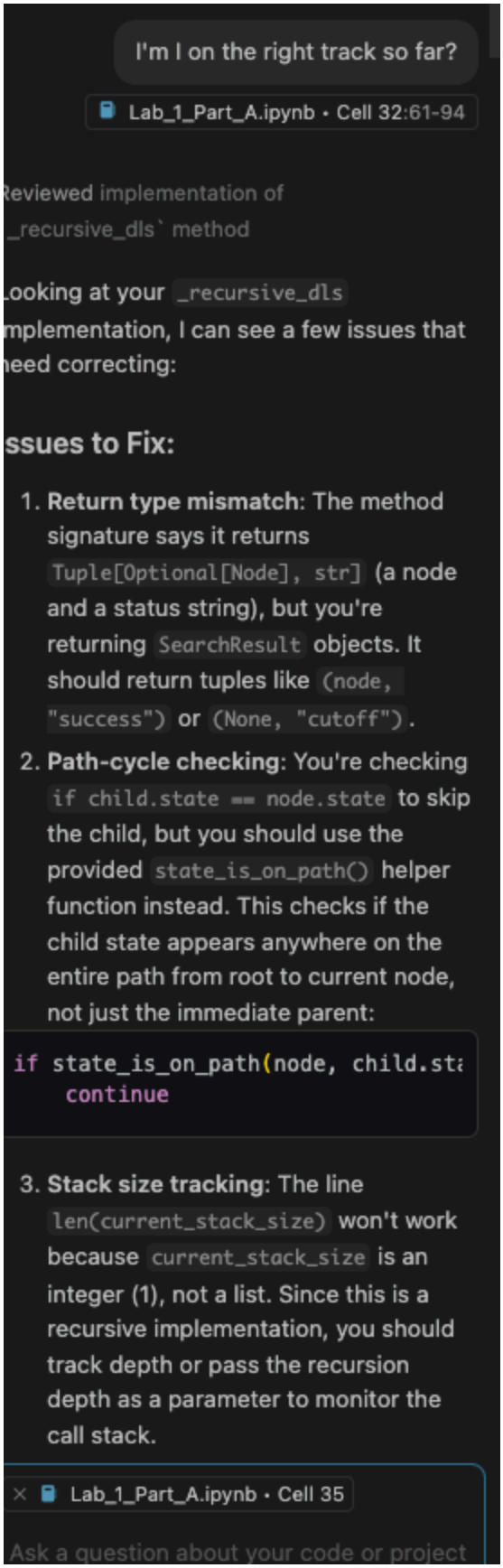
Let me look at your BFS code more closely. I can see several issues:

Reviewed Lab_1_Part_A.ipynb lines 714 to 777

Here are the main problems I see:

1. `return "failure"` **is inside the while loop** — It returns after expanding just the first node instead of continuing until the frontier is empty
2. `max_frontier_size = 0` **at the start of each iteration** — This resets it every time instead of tracking the maximum across all iterations
3. **Returns plain strings instead of** `SearchResult` **objects** — The function should return a `SearchResult`, not `"success"` or `"failure"`
4. `reached` **should be a set, not a list** — Better performance for membership checks
5. **There's also a bug in the** `expand` **method** — It calls `node(...)` instead of `Node(...)` (missing capital N)


How are search result objects returned?
Lab_1_Part_A.ipynb • Cell 0:12-40
Analyzed SearchResult object structure and BFS implementation issues
SearchResult is a dataclass that needs to be instantiated with specific parameters. Looking at the definition:
@dataclass
class SearchResult:
algorithm: str
status: str
solution: Optional[Node]
nodes_expanded: int
max_frontier_size: int
reached_count: int = 0
limit: Optional[int] = None
iterations: Optional[List[Dict
You need to return it like this:
return SearchResult(
algorithm="BFS",
status="success",
solution=child_node,
nodes_expanded=42,
max_frontier_size=15,
Lab_1_Part_A.ipynb • Cell 35
Ask a question about your code or project


I'm I on the right track so far?
Lab_1_Part_A.ipynb • Cell 32:61-94
eviewed implementation of
_recursive_dls` method
ooking at your _recursive_dls
mplementation, I can see a few issues that
eed correcting:
ssues to Fix:
1. Return type mismatch: The method signature says it returns Tuple[Optional[Node], str] (a node and a status string), but you're returning SearchResult objects. It should return tuples like (node, "success") or (None, "cutoff").
2. Path-cycle checking: You're checking if child.state == node.state to skip the child, but you should use the provided state_is_on_path() helper function instead. This checks if the child state appears anywhere on the entire path from root to current node, not just the immediate parent:
if state_is_on_path(node, child.st
continue
3. Stack size tracking: The line len(current_stack_size) won't work because current_stack_size is an integer (1), not a list. Since this is a recursive implementation, you should track depth or pass the recursion depth as a parameter to monitor the call stack.
Lab_1_Part_A.ipynb • Cell 35
Ask a question about your code or project

PROGRESS EVALUATION

Restore Checkpoint

for the stack size tracking, could I track depth eveytime recursive_dls is recursively called? So the depth is tied to node.depth?

Lab_1_Part_A.ipynb • Cell 32:63-89

Considered using `node.depth` for tracking recursion depth

Yes, that's a **much cleaner approach!** Since `node.depth` is automatically managed by the `Node` class and directly corresponds to the recursion depth, you can absolutely use it for stack size tracking.

Here's the idea:

```
def _recursive_dls(
    self,
    problem: Problem,
    node: Node,
    limit: int,
    metrics: Dict[str, int],
) -> Tuple[Optional[Node], str]:

    # Track recursion depth as pro
    metrics["max_stack_size"] = ma

    if problem.is_goal(node.state):
        return (node, "success")

    elif node.depth >= limit:
        return (None, "cutoff")

    else:
        metrics["nodes_expanded"] +
        cutoff_occurred = False

        for child in self.expand(p
            if state_is_on_path(no
                continue
```

PROGRESS EVALUATION

This works because each recursive call lands at a deeper depth, and `node.depth` automatically tracks that for you.

So given the node.depth + 1. Could you say dls is recursively called, the node that gets passed will automatically have a new depth per each recurse. So it means you'd increase the size to add on to the last depth?

Lab_1_Part_A.ipynb • Cell 10

Analyzed tracking recursion depth using node.depth + 1

Exactly right! You've got the key insight.

Here's why it works:

- **Initial call:** `node.depth = 0` → recursion depth = 1 → record `0 + 1 = 1` ✓
- **First recursive call:** `node.depth = 1` → recursion depth = 2 → record `1 + 1 = 2` ✓
- **Second recursive call:** `node.depth = 2` → recursion depth = 3 → record `2 + 1 = 3` ✓

Since `Node.__post_init__` automatically increments depth for each child, `node.depth` perfectly mirrors the recursion depth.

And yes—you update the **max** whenever you see a deeper level:

PROGRESS EVALUATION

Restore Checkpoint

I get bug 2 and bug 3 but not bug 1, If a a node is being expanded, is the parent reference to the node that gets passed before the new node gets the yield?

Lab_1_Part_A.ipynb • Cell 32:49-76

Considered parent reference for expanded node children

Good question! Let me clarify bug 1:

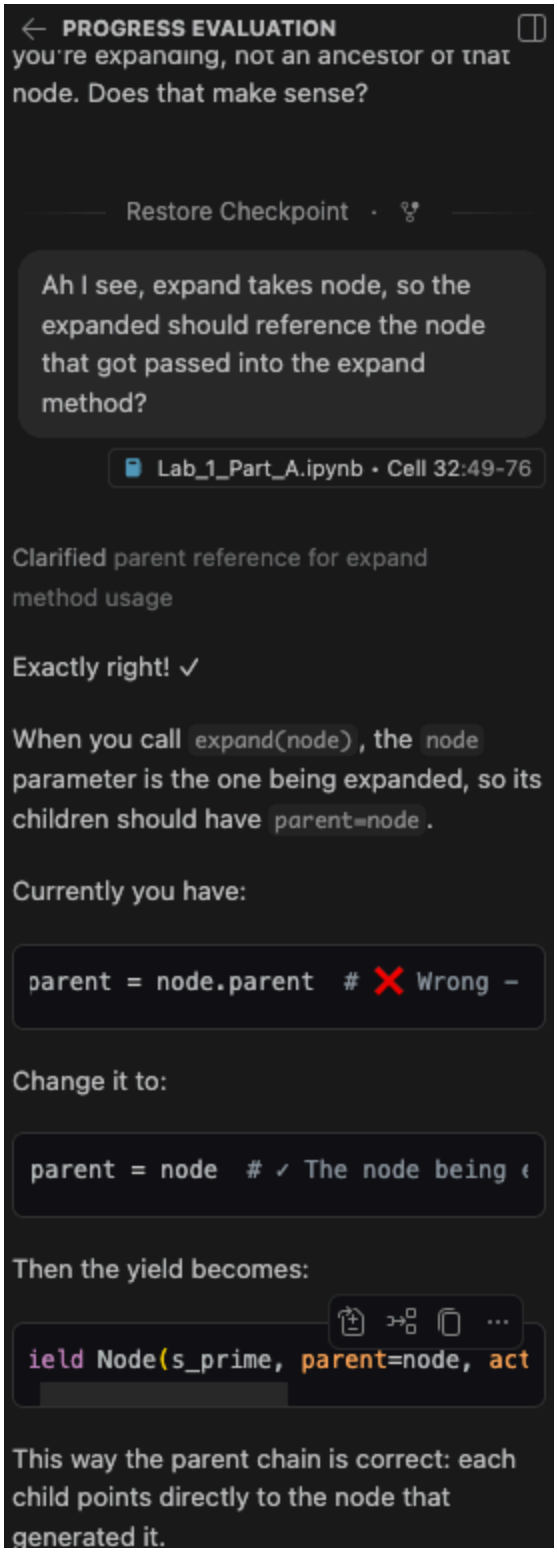
When you **expand** a node, you're creating its **children**. Each child's parent should point to **the node being expanded** (the one passed as parameter), not to that node's parent.

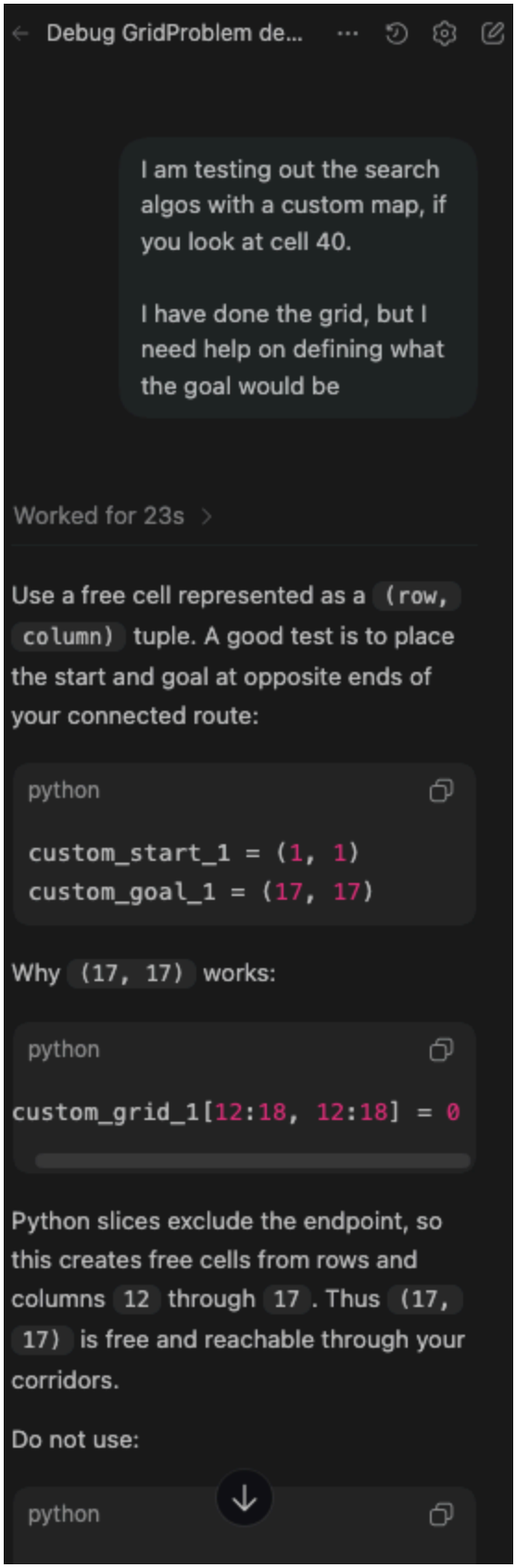
Think of it this way:

```
        grandparent
           |
        parent_node (this is what
         /  |  \
       c1  c2  c3  (these children
```

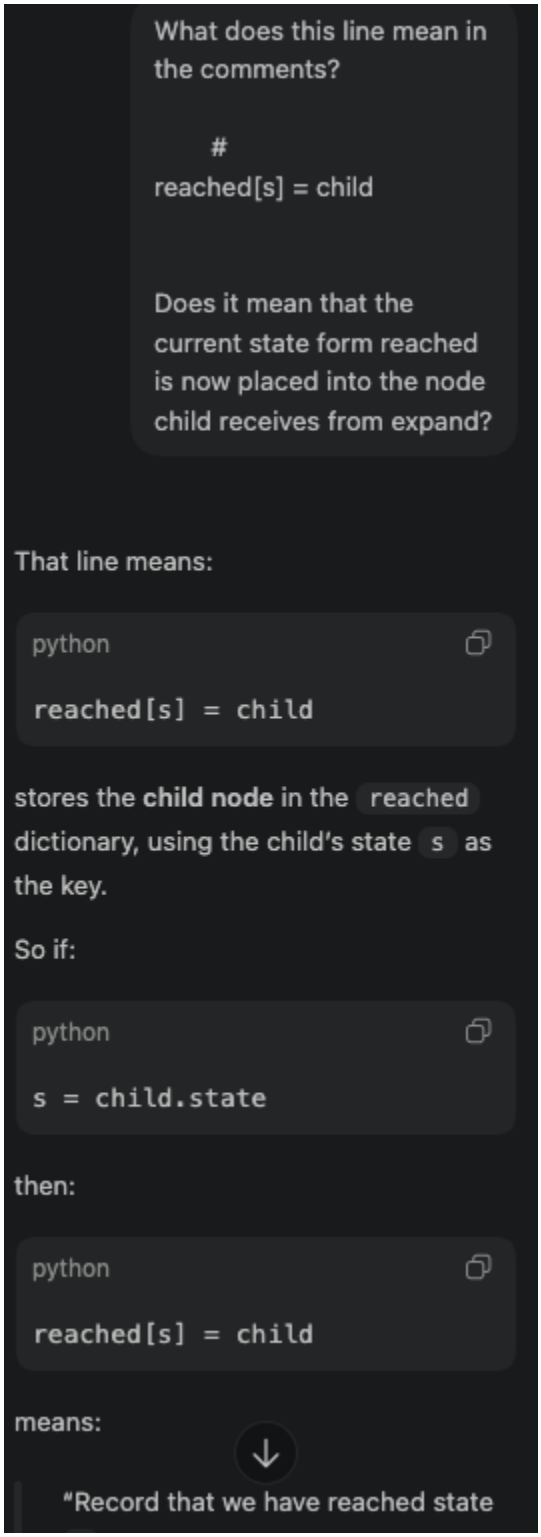
When you call `expand(parent_node)`, the children c1, c2, c3 should have:

- `parent = parent_node` ✓ (the node being expanded)
- `parent = parent_node.parent` ❌ (would skip a generation!)


PROGRESS EVALUATION
you're expanding, not an ancestor of that node. Does that make sense?
Restore Checkpoint
Ah I see, expand takes node, so the expanded should reference the node that got passed into the expand method?
Lab_1_Part_A.ipynb • Cell 32:49-76
Clarified parent reference for expand method usage
Exactly right! ✓
When you call expand(node), the node parameter is the one being expanded, so its children should have parent=node.
Currently you have:
parent = node.parent # ❌ Wrong –
Change it to:
parent = node # ✓ The node being e
Then the yield becomes:
ield Node(s_prime, parent=node, act
This way the parent chain is correct: each child points directly to the node that generated it.


Debug GridProblem de...
I am testing out the search algos with a custom map, if you look at cell 40.
I have done the grid, but I need help on defining what the goal would be
Worked for 23s
Use a free cell represented as a (row, column) tuple. A good test is to place the start and goal at opposite ends of your connected route:
python
custom_start_1 = (1, 1)
custom_goal_1 = (17, 17)
Why (17, 17) works:
python
custom_grid_1[12:18, 12:18] = 0
Python slices exclude the endpoint, so this creates free cells from rows and columns 12 through 17. Thus (17, 17) is free and reachable through your corridors.
Do not use:
python

Part B


What does this line mean in the comments?
#
reached[s] = child
Does it mean that the current state form reached is now placed into the node child receives from expand?
That line means:
python
reached[s] = child
stores the child node in the reached dictionary, using the child's state s as the key.
So if:
python
s = child.state
then:
python
reached[s] = child
means:
"Record that we have reached state

Quick question, do I have to return a SearchResult object for all the algorithms? Just like the Best first search one?

Yes. In this notebook design, every algorithm's `search(...)` method should return a `SearchResult` object.

That keeps the rest of the notebook consistent, especially helper functions like:

```python
show_results(results)
result_to_row(result)
plot_path(..., path=result.path
```

So even if the algorithm finds a `Node`, it should wrap it:

```python
return SearchResult(
    algorithm=algorithm,
    status="success",
    solution=node,
    nodes_expanded=nodes_expand
    max_frontier_size=max_front
    reached_count=len(reached),
)
```

And for failure:

Wait, the priority queue, which in this case a min heap, has the node with the lowest hueristic at the top?

Yes.

In this notebook, `PriorityQueue` is a **min-heap**, so the item with the **lowest priority value** is popped first.

For **Greedy Best-First Search**, the priority is:

```python
f(n) = h(n)
```

So the node with the **lowest heuristic estimate** is at the front of the search.

Meaning:

> Greedy expands the node that appears closest to the goal according to the heuristic.

Example:

```python
frontier.push(5, node_a)
frontier.push(2, node_b)
frontier.push(8, node_c)
```

Then:

python

What would the method signature for the f-limited search be?

A good method signature would be:

python

```
def _f_limited_search(
    self,
    node: Node,
    problem: Problem,
    limit: float,
    metrics: Dict[str, int],
) -> Tuple[Optional[Node], floa
```

Meaning:

- `node` : the current search node
- `problem` : the grid problem
- `limit` : the current f-cost limit
- `metrics` : dictionary for tracking `nodes_expanded` and max recursion depth/stack size

It returns:

python

```
(result, new_f)
```

Where:

- `result` is a [illegible] if the goal was found, otherwise `None`
- [illegible]

For the metrics, don't I need to update the max_frontier_size, to track depth?

Yes. For IDA*, `max_frontier_size` is being used to approximate the **maximum recursion stack depth**, not a real frontier size.

Since IDA* is recursive depth-first search, you can update it using the node depth:

python

```
metrics["max_frontier_size"] =
    metrics["max_frontier_size"
    node.depth + 1,
)
```

Use `node.depth + 1` because the root has `depth == 0`, but the stack contains one node at the root.

Put that near the start of `_f_limited_search`, before returning for cutoff or goal:

python

```
metrics["max_frontier_size"] =
    metrics["max_frontier_size"
    node.depth + 1,
)
```

```
f = node.path_cost + self.heuri
```

The thing is that metrics has not been defined at all, does it mean I have to create the metrics dict, then pass it to f limited search? Or I make it an optional parameter? but step 3 requires you to track both nodes expanded and the max frontier size

Yes, create the `metrics` dictionary inside `search()`, then pass it into `_f_limited_search()`.

Do not make it optional here. Since the requirement says to track `nodes_expanded` and `max_frontier_size`, make it explicit.

Inside `search()`:

python

```python
metrics = {
    "nodes_expanded": 0,
    "max_frontier_size": 1,
}
```

Then pass it into each IDA* iteration:

python

```python
result, next_limit = self._f_li
    initial_node,
    problem,
    limit,
    metrics,
```

I have to design two additional maps now. I will use numpy just like how I used it for part A

That is fine. For Part B, at least one of the two should be a `WeightedGridProblem` with meaningful terrain costs.

A good setup is:

Map 1: heuristic trap
Use obstacles to create a U-shape where the goal is visually close, but the opening is away from the goal. This tests Greedy.

Map 2: cheap long route vs expensive short route
Use terrain costs so the direct path is short but expensive, while a longer detour is cheaper. This tests UCS/A* against Greedy/Weighted A*.

With NumPy, the pattern is:

python

```
custom_grid_1 = np.zeros((15, 1
custom_costs_1 = np.ones((15, 1
```

Then add obstacles:

python

```
custom_grid_1[3:12, 7] = 1
```

And terrain costs:

Need a check from the top, apart from the reflection questions. I want to be sure that's the only thing left

Worked for 49s >

Not quite only reflections. I'd fix/check these before calling it done:

1. **`BestFirstSearch.nodes_expanded` is being counted in the wrong place.** Right now it increments inside the child loop, so it counts generated children, not expanded nodes. Move it to just before the `for child in self.expand(...)` loop:

```python
nodes_expanded += 1

for child in self.expand(proble
    state = child.state
    ...
```

and remove the current `nodes_expanded += 1` inside that loop. Then rerun all result tables because node counts will change.

2. **IDA* needs a cutoff return after the `for range(max_iterations)` loop.** If max iterations are exhausted, your `search()` currently falls through and returns `No`. Add: